

EmpDiary HTTPS and SSL Implementation Report

System: EmpDiary Employee Management System **Public domain:** empdiary.raccoon-ai.io **Server public IP:** 157.245.195.23 **Implementation date:** August 8, 2026 **Certificate authority:** Let's Encrypt **Reverse proxy:** Nginx

1. Executive summary

EmpDiary was originally accessed directly through unencrypted HTTP ports:

- Frontend: `http://157.245.195.23:3001`
- Backend: `http://157.245.195.23:8001`

The VM also hosts another project, so the existing application ports could not be replaced or reused. HTTPS was implemented without changing those ports and without changing the other project's Nginx configuration, certificate, or containers.

The final public architecture uses Nginx as a reverse proxy:

- `https://empdiary.raccoon-ai.io/` forwards to the EmpDiary frontend on `127.0.0.1:3001`.
- `https://empdiary.raccoon-ai.io/api/` forwards to FastAPI on `127.0.0.1:8001`.
- Requests to HTTP are redirected to HTTPS.
- A Let's Encrypt certificate was issued and automatic renewal was configured.

Several application and infrastructure problems appeared during the implementation. These included a nonstandard Nginx include layout, a certificate installation failure, Vite hostname rejection, incomplete JavaScript responses, stale Docker environment variables, browser mixed-content blocking, and incorrect FastAPI redirects. Each issue was diagnosed from configuration, response headers, browser messages, or server logs and corrected as described below.

2. Original architecture

The Docker Compose configuration exposed these EmpDiary services:

Service	Container port	VM host port	Original public use
React/Vite frontend	5173	3001	<code>`http://157.245.195.23:3001`</code>
FastAPI backend	8000	8001	<code>`http://157.245.195.23:8001`</code>
PostgreSQL	5432	5435	Database connection

The other application used different ports, including frontend port 3100 and backend port 8100. These ports and its domain, `theaisle.raccoon-ai.io`, were preserved.

Different applications do not require different public HTTPS ports. Nginx can listen once on ports 80 and 443 and select the correct application from the requested hostname:

<code>theaisle.raccoon-ai.io</code>	-> existing project
<code>empdiary.raccoon-ai.io</code>	-> EmpDiary

EmpDiary's ports 3001 and 8001 therefore remained unchanged behind Nginx.

3. DNS prerequisite

The selected domain was:

```
empdiary.raccoon-ai.io
```

Its public DNS A record was checked and resolved to:

```
empdiary.raccoon-ai.io -> 157.245.195.23
```

This check was required because Let's Encrypt must connect to the domain and confirm that it reaches the server requesting the certificate. Certificate issuance should not be attempted before DNS points to the correct VM.

4. Nginx discovery and multi-project isolation

The VM already contained:

- Nginx
- Certbot
- An existing HTTPS site for `theaisle.raccoon-ai.io`
- A valid certificate configuration for the other project

The existing project used:

```
Frontend -> 127.0.0.1:3100  
Backend  -> 127.0.0.1:8100
```

A separate EmpDiary configuration was prepared rather than modifying the other project's server block.

The project-owned copy of the Nginx configuration is located at:

```
deploy/nginx/empdiary.raccoon-ai.io
```

The deployed copy is located at:

```
/etc/nginx/sites-available/empdiary.raccoon-ai.io
```

The active include is:

```
/etc/nginx/conf.d/empdiary.conf
```

5. Certificate installation

Certbot requested a certificate for only the new hostname:

```
empdiary.raccoon-ai.io
```

The certificate and private key were stored at:

```
/etc/letsencrypt/live/empdiary.raccoon-ai.io/fullchain.pem  
/etc/letsencrypt/live/empdiary.raccoon-ai.io/privkey.pem
```

Certificate details observed after installation:

```
Subject:   CN = empdiary.raccoon-ai.io  
Valid from: August 8, 2026  
Expires:   November 6, 2026
```

Certbot reported that a scheduled task was configured to renew the certificate automatically.

Nginx now redirects:

```
http://empdiary.raccoon-ai.io
```

to:

```
https://empdiary.raccoon-ai.io
```

6. Complete error and resolution record

6.1 Error: Certbot issued the certificate but could not install it

Observed message

```
Successfully received certificate.  
Could not install certificate.  
Could not automatically find a matching server block for  
empdiary.raccoon-ai.io.
```

Why it happened

The EmpDiary configuration was initially enabled using the standard Ubuntu directory:

```
/etc/nginx/sites-enabled/
```

However, this VM's `/etc/nginx/nginx.conf` did not include `sites-enabled`. It loaded only:

```
include /etc/nginx/conf.d/*.conf;
```

The configuration file existed, but the running Nginx configuration could not see it. Certbot therefore could not find a matching active `server_name` block.

Solution

The EmpDiary configuration was linked into the directory actually loaded by this VM:

```
/etc/nginx/conf.d/empdiary.conf
```

Nginx validation was run again, and Certbot was instructed to install the already-issued certificate.

Result

```
Successfully deployed certificate for empdiary.raccoon-ai.io
```

This problem did not invalidate the certificate. It only prevented the first automatic Nginx installation attempt.

6.2 Error: HTTPS initially showed a certificate hostname mismatch

Observed message

```
SSL: no alternative certificate subject name matches target host name  
'empdiary.raccoon-ai.io'
```

Why it happened

Before the EmpDiary Nginx TLS server block was active, HTTPS traffic reached the server's existing default TLS configuration. That configuration presented the other project's certificate, which did not include `empdiary.raccoon-ai.io`.

Solution

The new server block was activated through `conf.d`, and the EmpDiary certificate was attached to it.

Verification

The certificate subject became:

```
CN = empdiary.raccoon-ai.io
```

6.3 Error: Frontend returned HTTP 403

Observed behavior

The API worked, but the frontend domain returned:

```
403 Forbidden
```

The response body stated:

```
Blocked request. This host ("empdiary.raccoon-ai.io") is not allowed.
```

Why it happened

The frontend was running Vite's development server. Vite protects against unexpected hostnames and did not yet trust the new public domain.

Solution

The following setting was added to `vite.config.js`:

```
allowedHosts: ["empdiary.raccoon-ai.io"]
```

Only the EmpDiary frontend container was restarted.

Verification

```
website_https=200  
api_https=200
```

6.4 Error: Blank page and ERR_CONTENT_LENGTH_MISMATCH

Observed behavior

The browser displayed a blank page. Developer Tools reported failures for files such as:

```
/@vite/client  
/@react-refresh  
/node_modules/.vite/deps/react-dom_client.js
```

The browser error was:

```
net::ERR_CONTENT_LENGTH_MISMATCH
```

Server evidence

Nginx logged:

```
open() "/var/lib/nginx/proxy/..." failed (13: Permission denied)  
while reading upstream
```

Why it happened

Nginx attempted to buffer larger responses from the Vite server into temporary files under `/var/lib/nginx/proxy`. The Nginx worker could not access those temporary files. The browser received fewer bytes than the declared response length and rejected the incomplete JavaScript.

Solution

Response buffering was disabled only inside EmpDiary's frontend location:

```
proxy_buffering off;
```

Changing global directory permissions was avoided because that would have affected the entire Nginx installation and potentially the other project.

Verification

The exact resources that previously failed were downloaded again:

```
Page:          HTTP 200, 1,022 bytes  
Vite client:   HTTP 200, 204,363 bytes  
React DOM:     HTTP 200, 820,982 bytes
```

The page then rendered correctly.

6.5 Error: Login request was blocked as mixed content

Observed browser message

```
Mixed Content: The page at  
'https://empdiary.raccoon-ai.io/login' was loaded over HTTPS,  
but requested an insecure resource  
'http://157.245.195.23:8001/auth/login'.  
This request has been blocked.
```

Why it happened

The website was secure, but the frontend's API environment value still pointed to the old HTTP backend. Browsers do not allow an HTTPS page to send sensitive requests to an HTTP endpoint because the request could be intercepted or modified.

The `.env` file had been changed, but a Docker restart did not recreate the container. Docker Compose reads `env_file` values when a container is created. Restarting the existing container preserved its old environment.

The retained value was confirmed inside the container:

```
http://157.245.195.23:8001
```

Solution

The frontend container was recreated, not merely restarted:

```
docker compose up -d --force-recreate --no-deps frontend
```

The API value was later made protocol-independent:

```
VITE_API_URL=/api
```

Why `/api` is safer

The browser automatically combines the relative path with the current secure origin:

```
/api/auth/login
```

becomes:

```
https://empdiary.raccoon-ai.io/api/auth/login
```

This prevents a deployment-specific IP address or HTTP protocol from being compiled into the frontend.

6.6 Error: Authenticated pages still produced mixed-content errors

Observed browser messages

Pages such as Dashboard, Users, and Supervisors attempted requests like:

```
http://empdiary.raccoon-ai.io/users/
```

The browser blocked them, and the UI showed messages such as:

```
Could not load data.
```

Initial investigation

The JavaScript currently served by the VM was inspected. It correctly contained:

```
"VITE_API_URL": "/api"
```

The browser had previously used an older module, so EmpDiary's development responses were also marked as non-cacheable:

```
add_header Cache-Control "no-store" always;
```

That was helpful, but it exposed a second root cause involving FastAPI redirects.

6.7 Error: FastAPI trailing-slash redirects lost HTTPS and `/api`

Observed response

A request to:

```
https://empdiary.raccoon-ai.io/api/users
```

returned:

```
HTTP/1.1 307 Temporary Redirect
Location: http://empdiary.raccoon-ai.io/users/
```

The same problem occurred with `/api/projects`.

Why it happened

Some FastAPI routes are registered with a trailing slash. When the frontend requested the version without the slash, FastAPI generated a 307 Temporary Redirect to the normalized route.

Nginx intentionally removes `/api` before sending the request to FastAPI:

```
Public request: /api/users
Backend request: /users
```

FastAPI generated its redirect using the backend-visible route. The resulting redirect:

- Used `http` instead of the public `https` scheme.
- Did not contain the public `/api` prefix.

The browser blocked the HTTP redirect before the request could complete.

Solution

An EmpDiary-specific `proxy_redirect` rule was added:

```
proxy_redirect ~^https?://empdiary\.racoon-ai\.io/(.*)$ /api/$1;
```

This converts backend-generated redirects back into the correct public route.

Before

```
Location: http://empdiary.racoon-ai.io/users/
```

After

```
Location: https://empdiary.racoon-ai.io/api/users/
```

Verification

```
users final URL:
https://empdiary.racoon-ai.io/api/users/
```

```
projects final URL:
https://empdiary.racoon-ai.io/api/projects/
```

The unauthenticated command-line tests ended with HTTP 401, which was expected because no login token was supplied. The important result was that both requests reached FastAPI using the correct HTTPS `/api` URL.

7. Application code and configuration changes

7.1 Frontend environment

File:

```
intern-tracking-system-frontend/.env
```

Final API configuration:

```
VITE_API_URL=/api
```

7.2 Vite hostname configuration

File:

```
intern-tracking-system-frontend/vite.config.js
```

Added:

```
allowedHosts: ["empdiary.racoon-ai.io"]
```

7.3 Backend CORS configuration

File:

```
intern-tracking-system/app/main.py
```

Added trusted HTTPS origin:

```
"https://empdiary.raccoon-ai.io"
```

7.4 PDF verification URL

File:

```
intern-tracking-system/app/core/pdf_service.py
```

Changed from the insecure IP URL to:

```
https://empdiary.raccoon-ai.io/verify
```

New approval PDFs should therefore contain secure public verification links.

7.5 Nginx configuration

Project copy:

```
deploy/nginx/empdiary.raccoon-ai.io
```

Important behaviors in the final configuration:

- Proxies / to frontend port 3001.
- Proxies /api/ to backend port 8001.
- Removes /api/ before sending requests to FastAPI.
- Passes original host and forwarding headers.
- Supports Vite's WebSocket upgrade.
- Disables frontend proxy buffering.
- Prevents caching of development frontend modules.
- Corrects FastAPI redirects so they retain HTTPS and /api.
- Allows request bodies up to 25 MB for backend uploads.
- Uses the dedicated EmpDiary certificate.
- Redirects HTTP to HTTPS.

8. Final request flow

```
Internet user
|
| HTTPS :443
v
Nginx selects empdiary.raccoon-ai.io
|
+-- /, /login, /dashboard, static and Vite files
|
|      +--> http://127.0.0.1:3001
|      EmpDiary React/Vite frontend
|
+-- /api/auth, /api/users, /api/projects, etc.
|
|      +--> http://127.0.0.1:8001
|      EmpDiary FastAPI backend
|
+--> PostgreSQL
```

The internal Nginx-to-container connection uses HTTP on the VM. This is normal for a local reverse proxy. The connection exposed to users over the internet is encrypted with HTTPS.

9. Validation performed

The following checks were completed during the implementation:

1. DNS resolved `empdiary.raccoon-ai.io` to `157.245.195.23`.
2. Direct frontend health check returned HTTP 200.
3. Direct backend documentation check returned HTTP 200.
4. Nginx configuration passed `nginx -t` before activation.
5. HTTP redirected to HTTPS.
6. The certificate subject matched `empdiary.raccoon-ai.io`.
7. The certificate validity period was inspected.
8. HTTPS frontend returned HTTP 200 after allowing the Vite hostname.
9. Previously truncated JavaScript files downloaded completely.
10. `/api/docs` returned HTTP 200.
11. The frontend container's environment value was checked after recreation.
12. The served `apiClient.js` was inspected and contained `VITE_API_URL: "/api"`.
13. FastAPI slash redirects were verified to stay on HTTPS under `/api`.

10. Effect on the other project

The following existing-project resources were not changed:

- Domain `theaisle.raccoon-ai.io`
- Ports 3100 and 8100
- Existing Nginx server block
- Existing TLS certificate
- Existing project containers

The EmpDiary configuration has its own hostname, certificate, upstream ports, and Nginx file. Nginx can serve both projects through ports 80 and 443 because it selects the destination from the hostname.

11. Current final state

Public application URL:

`https://empdiary.raccoon-ai.io`

Public API prefix:

`https://empdiary.raccoon-ai.io/api`

Internal services remain:

Frontend: `127.0.0.1:3001` through Nginx
Backend: `127.0.0.1:8001` through Nginx

The browser-facing parts of the system use HTTPS, HTTP redirects to HTTPS, and Let's Encrypt renewal is scheduled.

12. Remaining production and security recommendations

HTTPS protects data in transit, but it does not by itself make the entire deployment production-ready. The following work is still recommended.

12.1 Replace the Vite development server

The frontend currently runs:

```
npm run dev
```

The logs showed automated requests attempting to access paths such as `.env`, `.git/config`, and system files. Vite rejected these requests, but a public production site should not expose a development server.

Recommended production design:

1. Run `npm run build`.
2. Produce the frontend `dist` directory.
3. Serve `dist` directly with Nginx or a small static web-server container.
4. Remove Vite HMR and development modules from production.

This would also remove the need for the Vite-specific hostname, WebSocket, buffering, and no-cache workarounds.

12.2 Remove Uvicorn reload mode

The backend Dockerfile currently starts Uvicorn with:

```
--reload
```

Reload mode is intended for development. Production should use a stable process configuration without automatic source watching.

12.3 Restrict direct host ports

The Docker ports should ideally bind only to localhost:

```
frontend:
  ports:
    - "127.0.0.1:3001:5173"

backend:
  ports:
    - "127.0.0.1:8001:8000"
```

That would ensure users cannot bypass Nginx and access the unencrypted application ports directly.

12.4 Remove public PostgreSQL exposure

The database currently has a host port mapping for 5435. If no external database client requires it, the entire database `ports` section should be removed. The backend can still connect to PostgreSQL through Docker's internal network.

If external access is genuinely required, it should be restricted by firewall rules to trusted source IP addresses and protected with database TLS and strong credentials.

12.5 Disable development-only backend routes

FastAPI includes debug routes when `ENVIRONMENT` defaults to `development`. Production should set:

```
ENVIRONMENT=production
```

and confirm debug endpoints are unavailable.

12.6 Test renewal

Automatic renewal was configured by Certbot. It should be tested with:

```
sudo certbot renew --dry-run
```

This test should be run by an administrator and checked for a successful result.

12.7 Remove obsolete HTTP origins after migration

After confirming that nobody uses the old IP-and-port frontend, remove the obsolete HTTP production origin from FastAPI's CORS list. Localhost origins may remain if they are still needed for developer machines.

13. Operational checks

Useful administrator commands include:

Validate Nginx:

```
sudo nginx -t
```

Reload a validated Nginx configuration:

```
sudo systemctl reload nginx
```

Show certificate information:

```
sudo certbot certificates
```

Test certificate renewal:

```
sudo certbot renew --dry-run
```

View EmpDiary container state:

```
docker compose ps
```

View frontend logs:

```
docker compose logs --tail=100 frontend
```

View backend logs:

```
docker compose logs --tail=100 backend
```

View Nginx errors:

```
sudo tail -n 100 /var/log/nginx/error.log
```

14. Conclusion

EmpDiary was migrated from direct, insecure IP-and-port access to a domain-based HTTPS architecture while preserving the VM's other project and the EmpDiary port assignments.

The completed work included DNS validation, a separate Nginx virtual host, Let's Encrypt certificate issuance, automatic renewal setup, HTTPS redirection, same-origin API proxying, frontend environment changes, CORS changes, secure PDF links, Vite hostname configuration, incomplete-response correction, cache control, and FastAPI redirect rewriting.

The application is now available at:

```
https://empdiary.raccoon-ai.io
```

The most important next phase is production hardening: replace Vite's development server, remove Uvicorn reload mode, bind internal ports to localhost, and stop publicly exposing PostgreSQL unless specifically required.